

Case Study: Inventory Management System

1. Introduction

Modern retail and distribution businesses often operate across multiple warehouses and rely on accurate, real-time stock information to fulfil orders, avoid stockouts, and control costs. This case study presents the design and development of an Inventory Management System (IMS) — a web and mobile-based application that enables warehouse staff, inventory managers, purchasing officers, and administrators to track products, manage stock movements, coordinate with suppliers, and generate operational reports.

The problem statement requires selecting a real-world case study, defining user stories, choosing an appropriate software development process model and programming language, and proposing an implementation and evaluation strategy. The IMS addresses common inefficiencies such as manual stock counts, delayed reordering, mismatched warehouse records, and lack of visibility into supplier performance, by digitizing and automating core inventory workflows.

Core Objectives

- Maintain accurate, real-time stock levels across multiple warehouses.
- Automate low-stock alerts and reorder recommendations.
- Streamline purchase order creation and supplier coordination.
- Provide traceable stock-in, stock-out, and transfer records.
- Generate reports for stock valuation, movement, and audits.

2. User Stories

User ID	User Story	Priority
US-01	As an inventory manager, I want to log in securely so that I can access the system.	High
US-02	As a warehouse staff member, I want to scan a barcode to update stock quantity.	High
US-03	As an inventory manager, I want to add new products with SKU, category, and reorder level.	High
US-04	As an inventory manager, I want to view real-time stock levels across warehouses.	High
US-05	As a system, I want to trigger a low-stock alert when quantity falls below reorder level.	High
US-06	As a purchasing officer, I want to create purchase orders to suppliers.	High
US-07	As a warehouse staff member, I want to record incoming stock against a purchase order.	High
US-08	As a warehouse staff member, I want to record stock issued to departments or customers.	High
US-09	As an inventory manager, I want to transfer stock between warehouses.	Medium
US-10	As an admin, I want to manage supplier records and contact details.	Medium
US-11	As a manager, I want to generate stock valuation and movement reports.	Medium
US-12	As an admin, I want to conduct periodic stock audits and reconcile discrepancies.	Medium

3. Software Development Process Model

Selected Model: Agile Scrum

Reason for Selection

- Requirements evolve as warehouse operations scale.
- Continuous feedback from warehouse and purchasing teams.
- Incremental delivery of modules (stock, purchasing, reporting).
- Faster development and release cycles.
- Easier to isolate and fix bugs per module.
- Well suited for web and mobile-based enterprise applications.

Sprint Plan

Sprint	Features
Sprint 1	User Authentication & Role Management
Sprint 2	Product & Category Master Setup
Sprint 3	Stock In / Stock Out Management
Sprint 4	Purchase Order & Supplier Management
Sprint 5	Warehouse Transfer & Barcode Integration
Sprint 6	Alerts, Reporting & Analytics Dashboard
Sprint 7	Testing & Deployment

4. Programming Language and Technologies

Backend

- Java (Spring Boot)

Reasons

- Secure and scalable
- Well suited for enterprise-level applications
- Strong REST API support
- Excellent database connectivity and transaction management

Frontend

- HTML5
- CSS3
- JavaScript
- React.js

Database

- MySQL

Other Technologies

- REST API
- JWT Authentication
- Barcode/RFID Scanner Integration
- Git
- Docker
- AWS Cloud

5. Implementation Strategy

Phase 1 — Requirement Analysis

- Gather requirements from warehouse and purchasing teams.
- Create Software Requirement Specification (SRS).
- Prepare use case diagrams.

Phase 2 — System Design

- Design database schema (products, stock, warehouses, suppliers).
- Design UI wireframes for dashboard and mobile scanning app.
- Create UML diagrams.
- Develop system architecture.

Phase 3 — Development

- Implement user authentication and role management.
- Develop product and category master modules.
- Build stock in/out and transfer functionality.
- Integrate barcode/RFID scanning.
- Build purchase order and supplier management.
- Build reporting and alerts dashboard.

Phase 4 — Testing

Unit Testing

- Login and role validation
- Stock quantity calculations
- Reorder level trigger logic

Integration Testing

- Purchase order + stock receipt
- Stock transfer + warehouse update

System Testing

- Complete application testing end-to-end

User Acceptance Testing (UAT)

- Verify business requirements with warehouse and purchasing staff

Phase 5 — Deployment

- Docker Container
- AWS EC2
- MySQL Database
- Nginx Web Server

6. Evaluation Strategy

Evaluation Parameter	Method
Functional Accuracy	Requirement verification against SRS
Performance	Load testing using JMeter for concurrent stock updates

Evaluation Parameter	Method
Security	Authentication, role-based access, and penetration testing
Reliability	Continuous monitoring and stock data consistency checks
Usability	User feedback survey from warehouse and admin staff
Scalability	Concurrent user and multi-warehouse load testing
Maintainability	Code review, modular design, and documentation

8. UML Diagrams

8.1 Use Case Diagram

The use case diagram below captures the primary interactions between the two main actor groups — Warehouse Staff, who perform day-to-day stock operations, and Admin/Manager, who configure and oversee the system — with the core functions of the Inventory Management System.

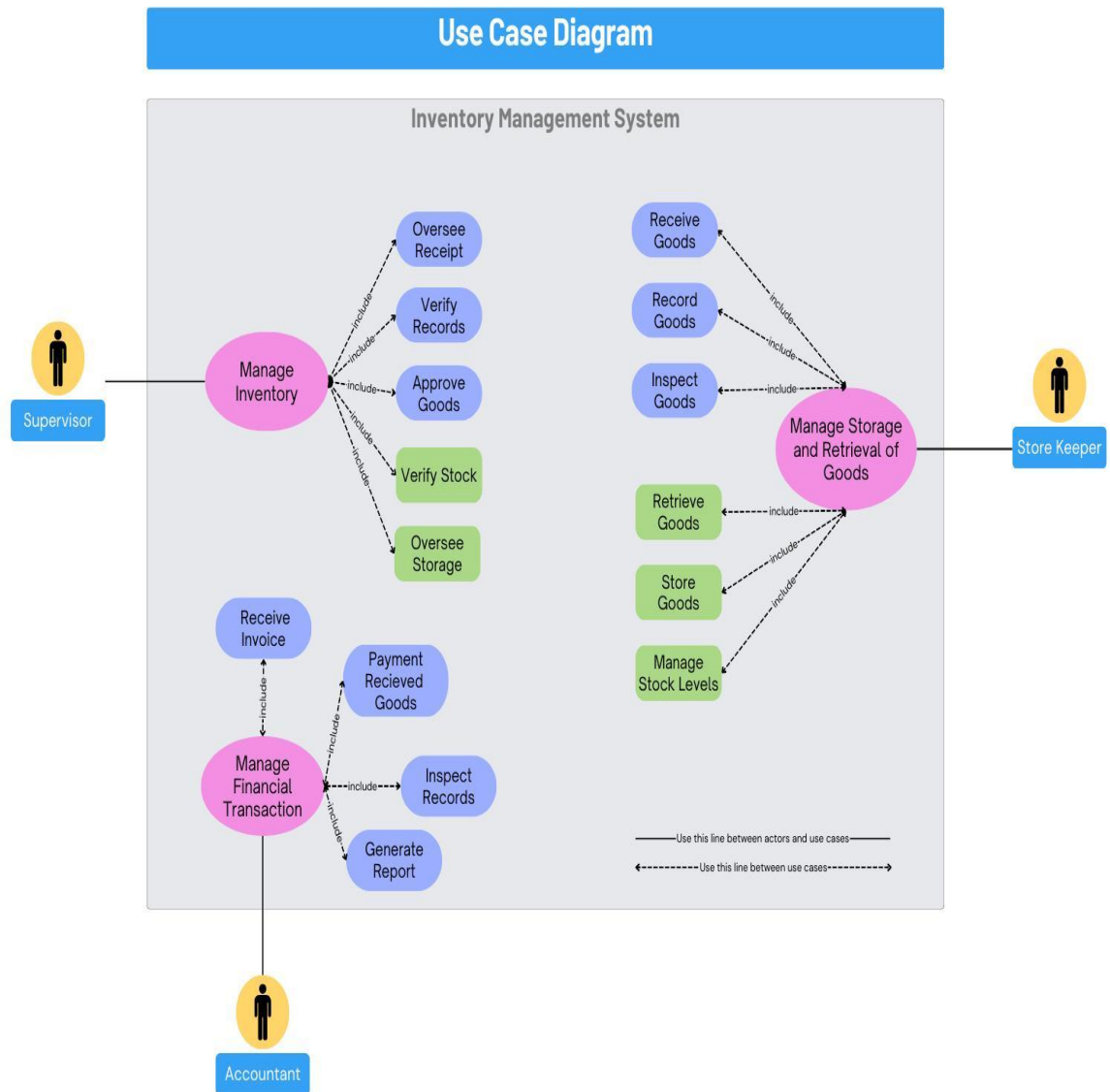
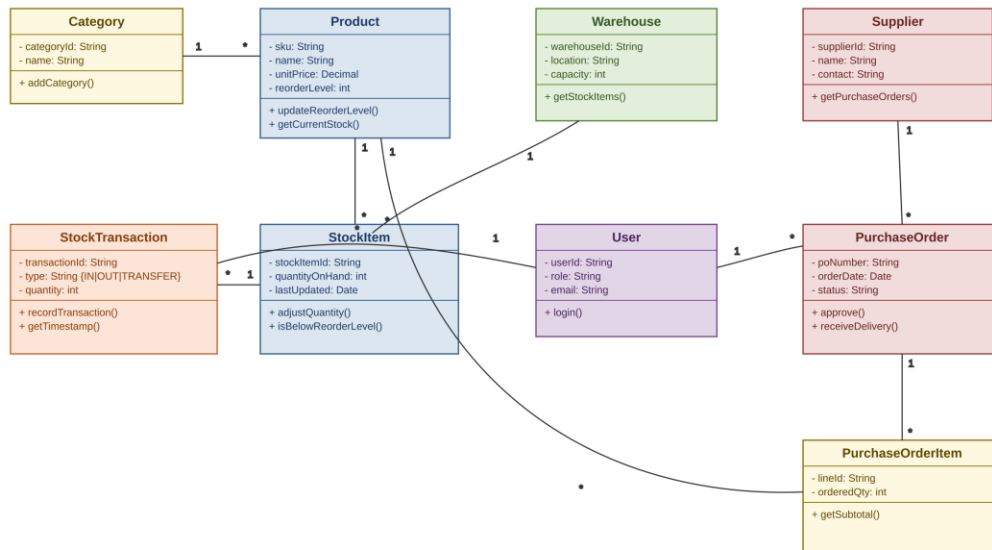


Figure 1: Use Case Diagram — Inventory Management System

8.2 Class Diagram

The class diagram below models the core domain entities of the system — Product, Category, Warehouse, StockItem, StockTransaction, Supplier, PurchaseOrder, PurchaseOrderItem, and User — along with their key attributes, operations, and relationships.



Class Diagram — Core domain model of the Inventory Management System

Figure 2: Class Diagram — Inventory Management System

9. Conclusion

The proposed Inventory Management System addresses the operational challenges of multi-warehouse stock control by combining a scalable Spring Boot/React architecture with an Agile Scrum delivery approach. The defined user stories, sprint plan, and UML models provide a clear blueprint for incremental development, while the phased implementation and evaluation strategy ensure that functional accuracy, performance, security, and usability are validated before deployment. Once implemented, the system is expected to reduce stockouts and overstocking, accelerate purchase order processing, and give stakeholders reliable, real-time visibility into inventory across the organization — forming a solid foundation for future enhancements such as demand forecasting and supplier performance analytics.